챕터 2. Docker 이해하기

다음 날 저녁, 오픈이는 커피 한 잔을 들고 노트북 앞에 앉았습니다. 전날 본 영상으로 컨테이너와 도커가 왜 필요한지 알게 됐습니다. 환경이 어디서나 같아야 한다는 것, 그 답이 컨테이너라는 것까지는 이해했습니다.

이제 남은 건 직접 실습해 보는 일이었습니다. 오늘의 목표는 하나였습니다.

Docker가 무엇이고 어떻게 돌아가는지, 개념과 기본 사용법을 손에 익힌다.

지금부터 도커가 컨테이너를 어떻게 만들고, 도커를 어떻게 사용하는지 알아보겠습니다.

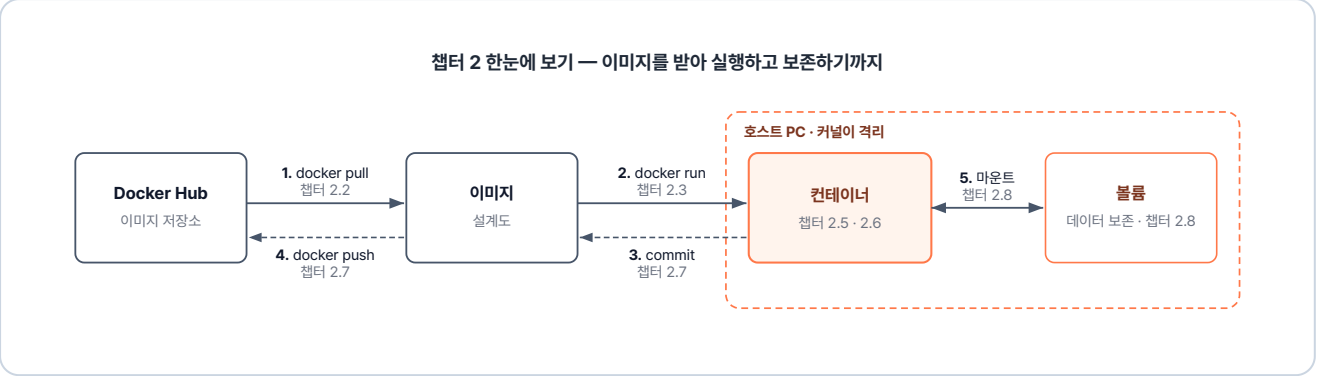


그림 2-1. 이번 챕터 실습 흐름 - 이미지를 받아 실행하고 보존하기까지

준비하기

Docker Desktop 설치

공식 사이트(https://www.docker.com/products/docker-desktop/)에서 OS에 맞는 설치 파일을 받아 설치합니다.

> **설치 전 확인 사항**
>
> - **Windows**: BIOS/UEFI에서 **하드웨어 가상화(Intel VT-x 또는 AMD-V)** 가 켜져 있어야 합니다. 자동으로 바꿀 수 없는 항목이라 설치 전에 직접 확인이 필요합니다. WSL2는 Docker Desktop 설치 과정에서 함께 자동 설치됩니다.
> - **macOS Apple Silicon(M1/M2/M3 등)**: 일부 AMD64 이미지 호환을 위해 **Rosetta 2** 설치를 권장합니다. 터미널에서 `softwareupdate --install-rosetta` 명령으로 설치할 수 있습니다.

2.1 가상화 - 가상머신과 컨테이너

2.1.1 한 주방, 네 명의 요리사

Docker와 컨테이너를 이해하려면 먼저 **가상화**를 알아야 합니다. 주방을 예로 들어 보겠습니다.

한 주방에 요리사 네 명이 있습니다. 네 명이 **동시에** 요리를 하려면 주방을 어떻게 꾸며야 할까요.

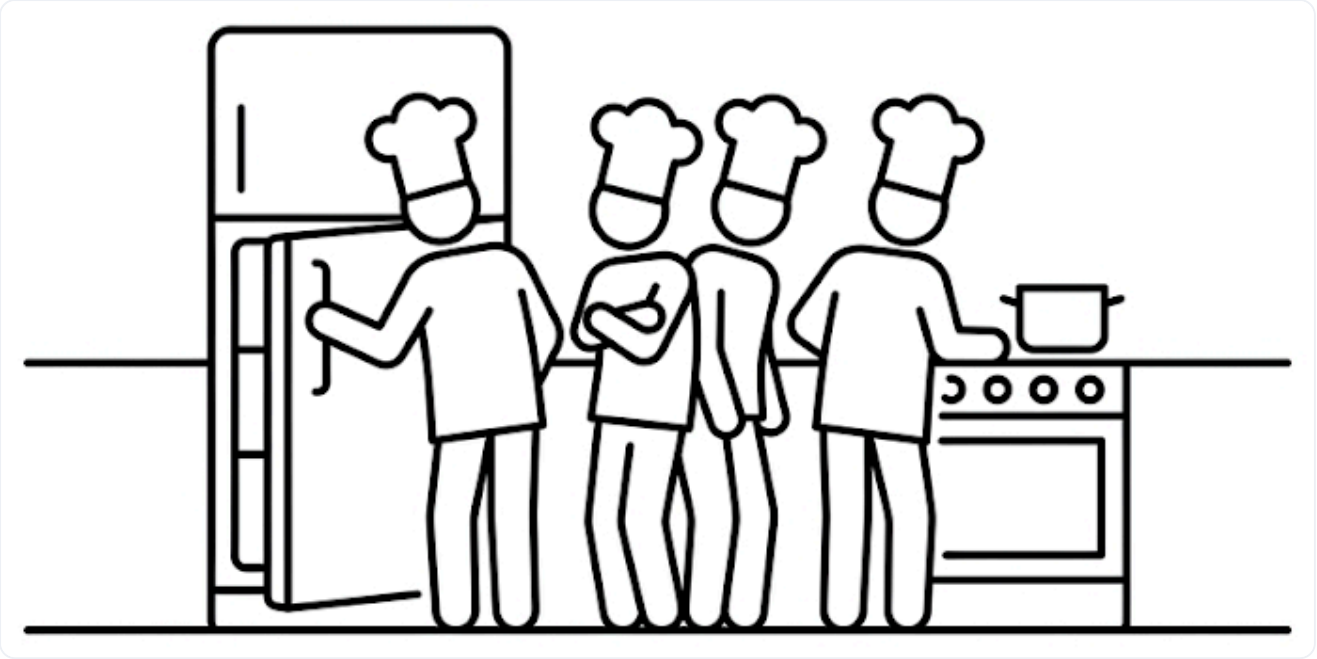


그림 2-2. 한 주방에 네 명의 요리사가 있는 상황

첫 번째 방식은 요리사마다 냉장고, 가스레인지, 조리대를 각자 한 세트씩 구매하는 방법입니다. 이렇게 하면 네 사람이 **완전히 독립된 환경**에서 요리하지만, 같은 설비를 네 번 갖춰야 하니 **비용이 많이 듭니다**.

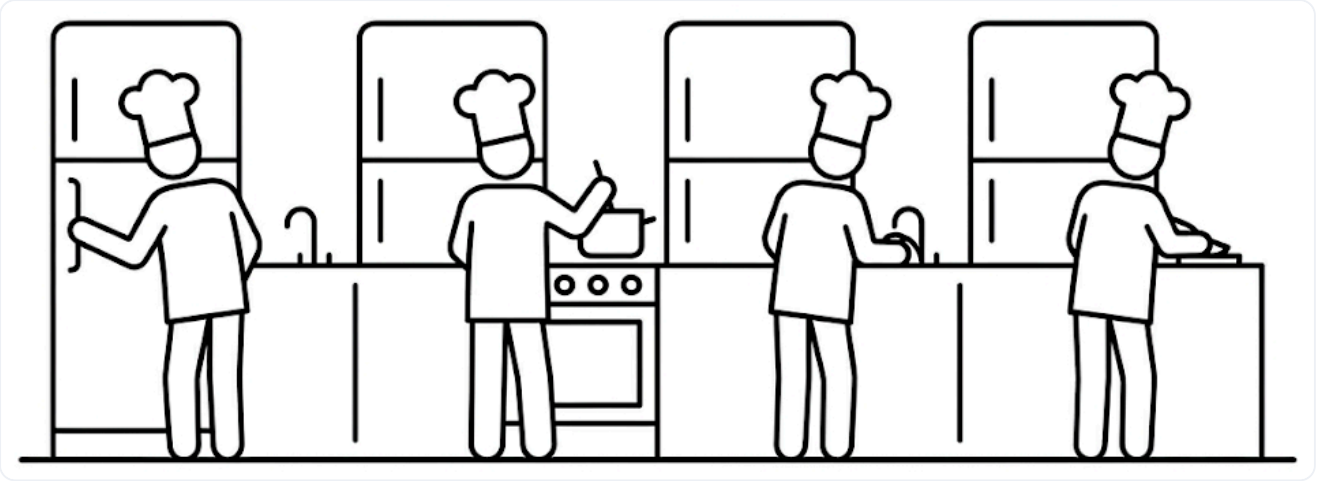


그림 2-3. 주방 설비를 통째로 복제하는 첫 번째 방식

두 번째 방식은 냉장고와 가스레인지 같은 공용 설비는 함께 쓰고, 칼·도마·조리대처럼 개인이 쓸 도구만 각자 챙기는 방법입니다. 그래서 설비를 더 늘리지 않고도 **공간을 효율적으로 나눠 쓸 수 있습니다**.

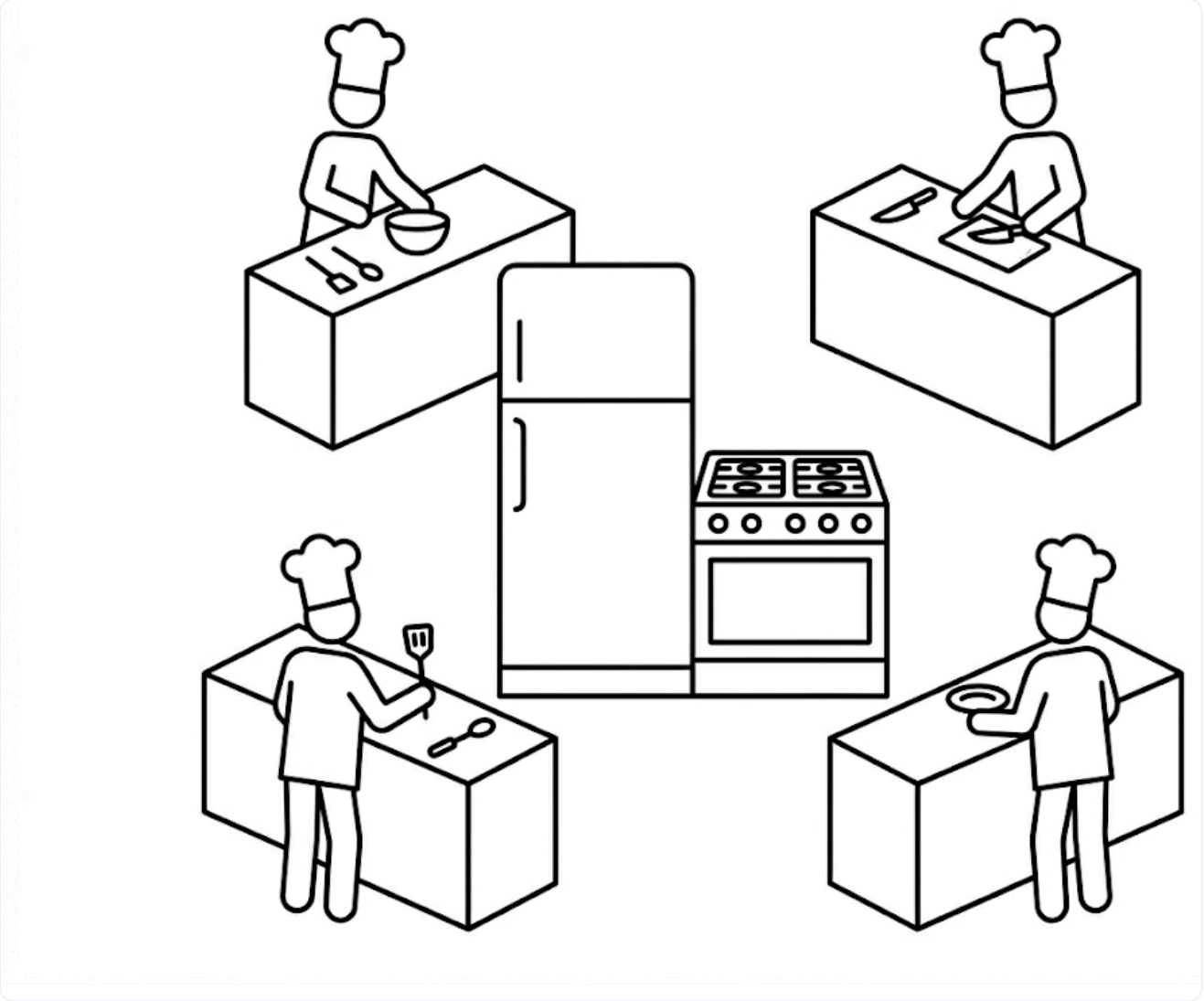


그림 2-4. 공용 설비는 공유하고 개별 공간만 쪼개는 두 번째 방식

이렇게 하나의 주방을 여러 명이 사용하는 것처럼, 하나의 컴퓨터를 여러 대처럼 나눠 쓰는 IT 기술을 **가상화(Virtualization)** 라고 부릅니다. 자원을 아끼면서도 서로 다른 환경을 안전하게 격리하는 것이 목적입니다.

가상화에는 크게 두 가지 방식이 있습니다. 환경을 통째로 복제하는 **하이퍼바이저 가상화**와, 공용 부분을 함께 쓰는 **컨테이너 가상화**입니다. **Docker는 이 컨테이너 가상화를 다루는 도구입니다.**

> **하이퍼바이저 가상화 vs 컨테이너 가상화**
> **하이퍼바이저 가상화(Hypervisor Virtualization)** 는 호스트 OS 위에 **또 다른 OS를 통째로 올립니다.** 이렇게 떠 있는 OS 한 대가 곧 **가상 머신(VM)** 입니다. 격리는 완전하지만 OS 전체가 새로 떠야 해서 무겁고 기동이 느립니다.
> 반면 **컨테이너 가상화(Container Virtualization)** 는 **호스트 OS의 커널을 공유**하고, 파일시스템·네트워크·프로세스 공간만 따로 둡니다. 그 안에서 도는 격리된 단위가 **컨테이너**입니다. 커널을 공유하므로 가볍고 빠르게 뜹니다.

'그러니까 도커는 컴퓨터 OS 하나를 여러 컨테이너가 같이 쓰면서도, 각자 독립된 공간에서 따로 돌게 만드는 거네.'

2.1.2 IT로 옮긴 구조

비유로 알아본 가상화를 실제 구조로 옮기면 다음과 같습니다.

주방의 공용 설비가 호스트 OS라면, 각자의 독립된 조리대는 컨테이너에 비유할 수 있습니다.

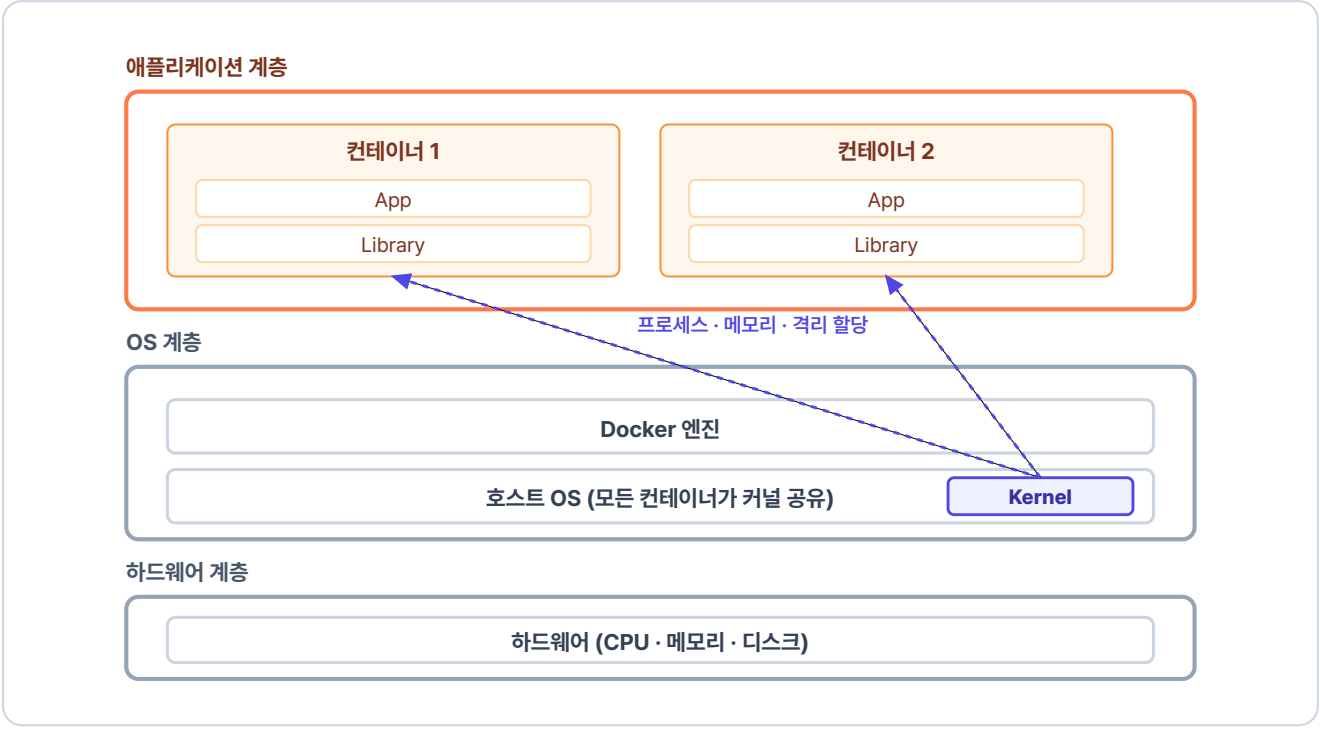


그림 2-5. 컨테이너 가상화의 전체 구조

컨테이너 가상화 구조는 크게 세 계층으로 나뉩니다. 맨 아래는 **하드웨어 계층**, 그 위는 호스트 OS와 Docker 엔진이 있는 **OS 계층**, 맨 위는 독립된 컨테이너들이 실행되는 **애플리케이션 계층**입니다. 각 컨테이너는 실행에 필요한 애플리케이션과 라이브러리를 독립적으로 갖추고 있습니다.

컨테이너마다 이런 독립을 가능하게 하는 것은 호스트 OS의 핵심인 **커널(Kernel)** 입니다. 주방장이 주방을 총괄하듯, 커널은 컨테이너마다 **독립된 프로세스**와 **격리된 메모리 공간**을 할당하고 관리합니다. 이때 호스트의 커널은 **하나뿐이고**, Docker의 모든 컨테이너가 그 **같은 리눅스 커널**을 공유합니다.

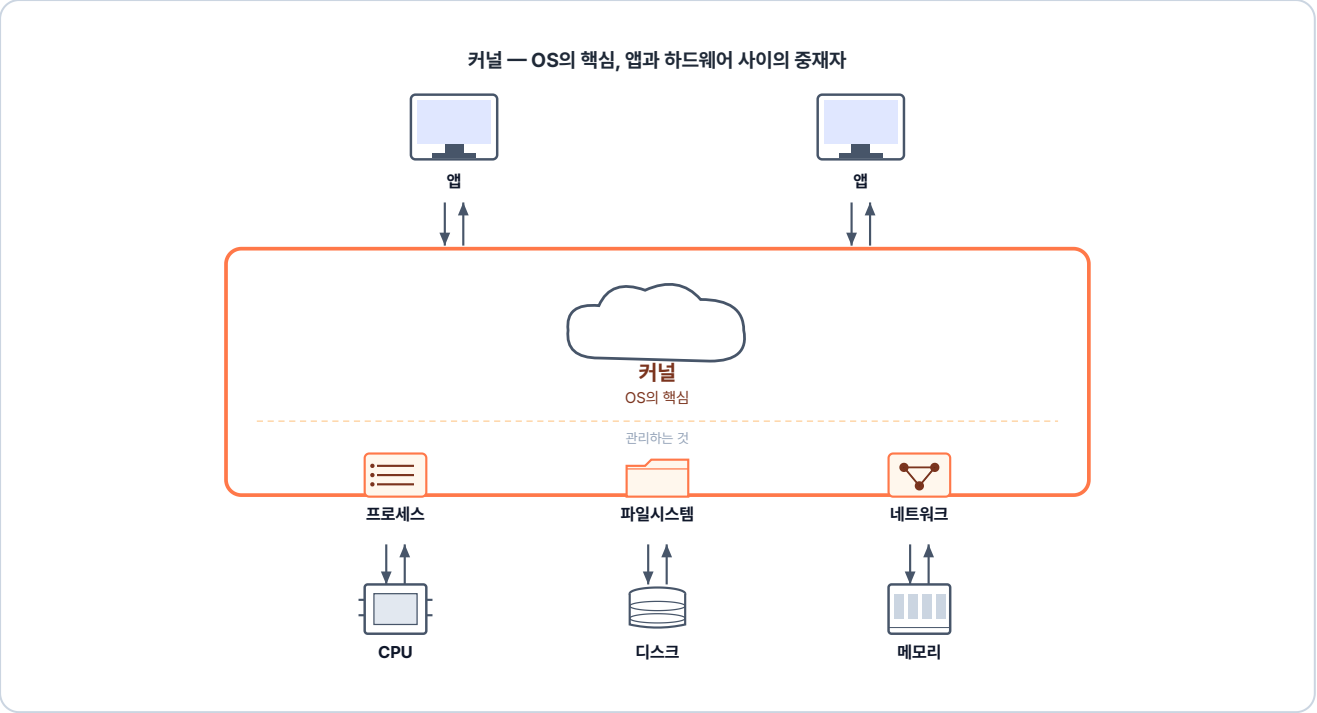


그림 2-6. 커널은 OS의 핵심으로, 앱과 하드웨어 사이에서 시스템 자원을 관리합니다

그래서 컨테이너들은 서로 다른 앱과 라이브러리를 가져도 충돌하지 않고, 한쪽이 런타임을 업그레이드해도 옆 컨테이너는 영향을 받지 않습니다.

> **컨테이너의 격리**
> 사실 컨테이너 안에서 도는 것은 처음부터 격리된 무언가가 아니라, 호스트 OS 위에서 다른 것들과 함께 도는 **평범한 프로세스**입니다. 그대로 두면 한 컨테이너의 프로세스가 다른 컨테이너의 파일을 들여다보거나, 두 컨테이너가 같은 포트를 두고 충돌할 수 있습니다. 그래서 **리눅스 커널**은 컨테이너마다 **파일시스템**(`/bin`, `/lib` 같은 폴더), **네트워크**(IP와 포트), **프로세스**(PID 번호) 세 가지 공간을 따로 만들어 줍니다. 커널이 이렇게 따로 만들어 주는 공간을 **네임스페이스(Namespace)** 라고 합니다. 컨테이너마다 자기 네임스페이스를 갖는 이 상태가 바로 **격리**입니다.

이 구조에서 **Docker 엔진**은 주문을 받는 매니저 역할을 합니다. 우리가 커널을 직접 제어하는 대신 Docker 엔진에게 명령을 내리면, 엔진이 그 명령을 커널에 전달합니다.

2.1.3 컨테이너가 만들어지는 과정

지금까지는 컨테이너가 **무엇인지**를 살펴봤습니다. 그렇다면 이 컨테이너는 **어떻게** 만들어질까요. 명령어를 실행했을 때 무엇이 어떤 순서로 일어나는지 따라가 보겠습니다.

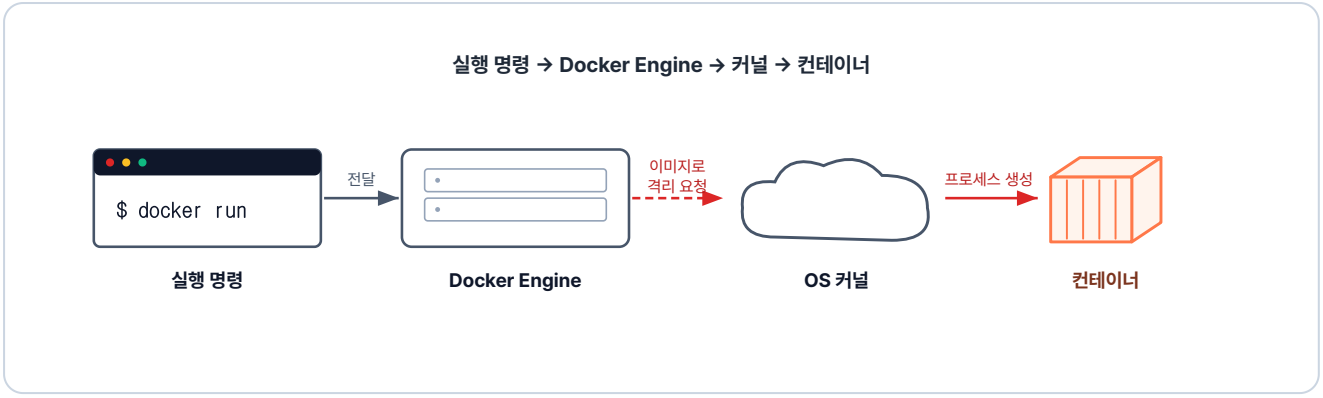


그림 2-7. 컨테이너 실행 명령이 커널의 격리 기능을 타고 컨테이너가 되는 흐름

사용자가 컨테이너 실행 명령을 내리면, **Docker 엔진**은 가장 먼저 로컬에 해당 이미지가 있는지 확인하고, 없으면 Docker Hub에서 자동으로 내려받습니다. 이미지가 준비되면 Docker 엔진은 그 이미지를 바탕으로 격리된 프로세스를 만들어 달라고 호스트 OS의 **커널**에 요청합니다. 요청을 받은 커널은 격리된 공간을 만든 뒤 그 안에서 새 프로세스를 띄웁니다.

이렇게 격리된 공간에서 실행되는 프로세스가 곧 컨테이너입니다.

'그럼 컨테이너를 만드는 재료인 이미지는 뭐지.'

2.2 이미지 - 컨테이너의 설계도

2.2.1 이미지란?

이미지(Image) 는 컨테이너가 무엇을 실행할지를 담은 **설계도**입니다. 이미지에는 무엇을 어떻게 실행할지가 모두 담겨 있고, 그 설계도를 실제로 실행한 것이 컨테이너입니다.

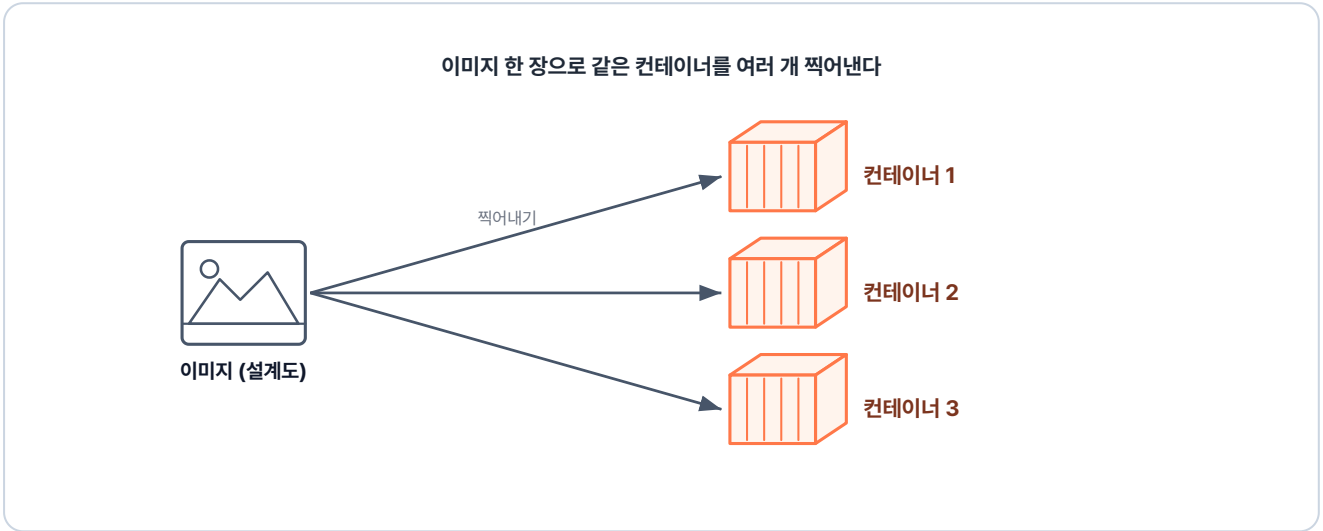


그림 2-8. 하나의 이미지로 여러 컨테이너를 찍어내는 구조

이미지는 **붕어빵 틀**과 같습니다. 붕어빵 틀 하나로 똑같은 붕어빵을 원하는 만큼 찍어내듯, 이미지 하나로 똑같은 컨테이너를 몇 개든 만들 수 있습니다. 같은 이미지로 만든 컨테이너는 어디서 실행하든 그 안의 환경이 똑같습니다. 이 덕분에 **내 PC에서 돌아가던 환경을 서버에 그대로 재현하는 일**이 가능해집니다.

2.2.2 이미지 레이어

이미지는 코드와 라이브러리, 설정을 하나로 묶은 패키지입니다. 하나의 큰 파일이 아니라 **여러 레이어가 층층이 쌓인 구조**입니다.

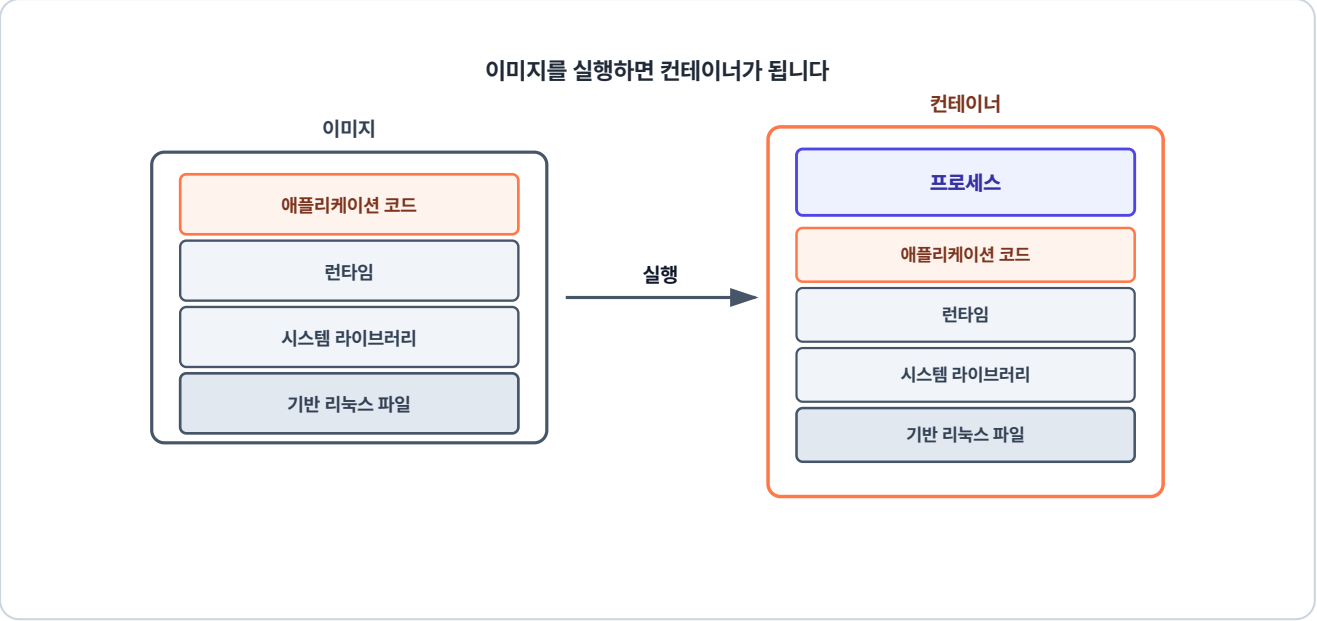


그림 2-9. 이미지를 실행하면 레이어 묶음 위에서 프로그램이 도는 컨테이너가 됩니다

맨 아래 세 레이어(**기반 리눅스 파일·시스템 라이브러리·런타임**)는 여러 앱이 공통으로 쓰는 **거의 바뀌지 않는** 부분이고, 맨 위 레이어(**애플리케이션 코드**)는 그 앱에서만 쓰는 **자주 바뀌는** 코드입니다. 그래서 코드만 바뀐 새 버전을 만들 때도 아래 세 레이어는 그대로 재사용하고, 바뀐 맨 위 레이어만 새로 만듭니다.

이미지를 실행하면, 층층이 쌓인 이 레이어들이 하나로 합쳐져 그 위에서 프로그램이 도는 **컨테이너가 됩니다**.

2.2.3 Docker Hub

이미지는 **Docker Hub**라는 공용 저장소에서 내려받을 수도 있고, 직접 만들어 올릴 수도 있습니다.

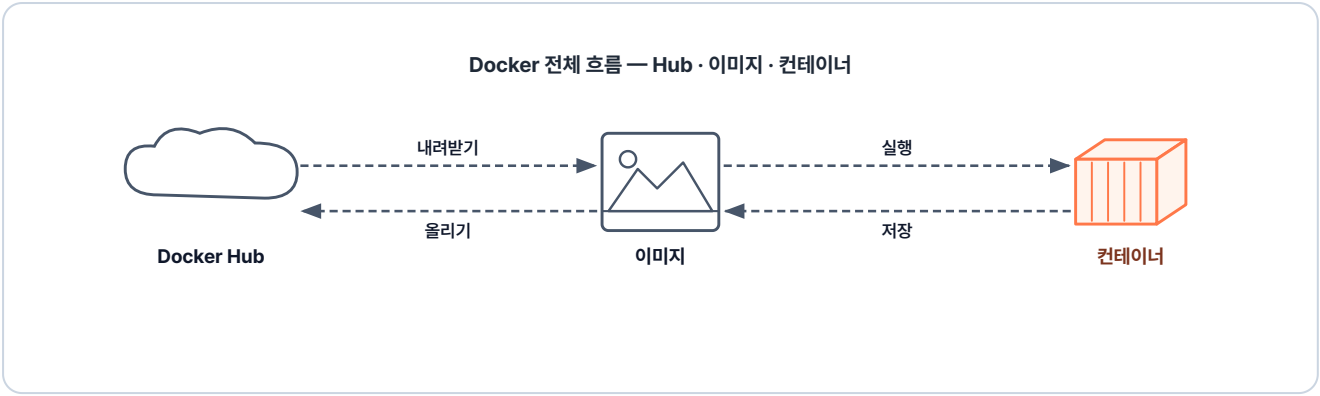


그림 2-10. Docker의 전체 흐름

Docker Hub를 활용하면 내 PC에서 세팅한 환경을 서버에서 재현하는 일도, 팀원끼리 환경을 공유하는 일도 가능합니다.

'도커 컨테이너와 이미지는 알았으니, 이제 직접 실행해 보자.'

2.3 Docker CLI - 첫 컨테이너 띄우기

이제부터는 터미널을 열고 `docker` 명령어를 직접 입력해 컨테이너를 띄워 보겠습니다.

2.3.1 docker pull: 이미지 가져오기

가장 먼저 Docker Hub에서 이미지를 내려받습니다.

터미널 **nginx 이미지 받기**

```
docker pull nginx   # nginx 이미지 다운로드
```

실행결과

```
$ docker pull nginx
Using default tag: latest
latest: Pulling from library/nginx
9baba07a35b6: Pull complete
ec781dde3f47: Pull complete
0289d65812c3: Pull complete
4174e33a2c9e: Pull complete
6b40784e4837: Pull complete
980067d12da2: Pull complete
f0b77348d9b0: Pull complete
00238b7dc6b2: Pull complete
Digest: sha256:9b3a2dde20b8a1b5e9f2c4a6d8e0b3f5a7c9e1b3d5f7a9b1c3e5f7a9b1c3e5f7
Status: Downloaded newer image for nginx:latest
docker.io/library/nginx:latest
$
```

그림 2-11. nginx 이미지 다운로드 결과

명령어 실행이 완료되면 로컬 저장소에 이미지가 저장됩니다. 다만 저장만 됐을 뿐, 아직 실행된 것은 아닙니다.

2.3.2 docker run: 컨테이너 띄우기

이어서 다운받은 이미지로 컨테이너를 실행해 봅니다.

터미널 **nginx 컨테이너 실행 (포그라운드)**

```
docker run nginx   # nginx 컨테이너 실행
```

실행 결과

```
$ docker run nginx
/docker-entrypoint.sh: /docker-entrypoint.d/ is not empty, will attempt to perform
configuration
/docker-entrypoint.sh: Looking for shell scripts in /docker-entrypoint.d/
/docker-entrypoint.sh: Launching /docker-entrypoint.d/10-listen-on-ipv6-by-default.sh
10-listen-on-ipv6-by-default.sh: info: Getting the checksum of /etc/nginx/conf.d/default.conf
10-listen-on-ipv6-by-default.sh: info: Enabled listen on IPv6 in
/etc/nginx/conf.d/default.conf
/docker-entrypoint.sh: Sourcing /docker-entrypoint.d/15-local-resolvers.envsh
/docker-entrypoint.sh: Launching /docker-entrypoint.d/20-envsubst-on-templates.sh
/docker-entrypoint.sh: Launching /docker-entrypoint.d/30-tune-worker-processes.sh
/docker-entrypoint.sh: Configuration complete; ready for start up
2026/03/17 10:38:22 [notice] 1#1: using the "epoll" event method
```

그림 2-12. nginx 컨테이너 실행

명령어를 실행하자 터미널이 멈추고 아무 반응이 없습니다.

이는 컨테이너가 **포그라운드** 상태로 터미널을 점유하고 있기 때문입니다. 통화 중일 때는 다른 전화를 받을 수 없는 것과 같습니다.

> **포그라운드(Foreground)**: 프로세스가 터미널을 독점하는 상태입니다. 프로세스가 끝나거나 강제 종료될 때까지 그 터미널로 다른 명령을 내릴 수 없습니다.

우선 터미널에 `CTRL + C`를 입력해 빠져나옵니다.

2.3.3 docker run -d: 백그라운드로 띄우기

'컨테이너는 띄운 채로 터미널도 같이 쓰려면 어떻게 해야 하지?'

이 문제를 풀어 주는 게 `-d` 옵션입니다. `-d` 옵션을 사용하면 백그라운드 상태에서 프로세스를 실행할 수 있습니다.

터미널 **백그라운드로 컨테이너 띄우기**

```
docker run -d nginx   # -d: detached, 백그라운드 실행
```

실행 결과

```
$ docker run -d nginx
4932c46c66589dc651efd5e50a8773d6d077f8a1e005f177385f8e0cc115ad7d
```

그림 2-13. 백그라운드 실행 결과

이번에는 컨테이너 ID가 찍히고 곧바로 프롬프트가 돌아왔습니다. 방금 프롬프트가 돌아오지 않던 것과 달리, `-d`로 띄우면 컨테이너는 백그라운드에서 돌고 터미널은 그대로 쓸 수 있습니다.

> **백그라운드(Background)**: 프로세스가 터미널을 점유하지 않고 뒤에서 실행되는 상태입니다. 프로세스가 작동하는 중에도 같은 터미널 창에서 다른 명령어를 계속 입력하고 실행할 수 있습니다.

'터미널을 계속 쓰려면 백그라운드로 실행하면 되겠네.'

2.3.4 docker ps: 실행 중인 컨테이너 확인

백그라운드에서 돌고 있는 컨테이너가 실제로 살아 있는지 확인합니다.

터미널 **실행 중 컨테이너 확인**

```
docker ps   # 실행 중인 컨테이너 목록
```

실행 결과

```
$ docker ps
CONTAINER ID IMAGE COMMAND CREATED STATUS PORTS NAMES
4932c46c6658 nginx "/docker-entrypoint...." 1 second ago Up Less than a second 80/tcp
dazzling_carso
```

그림 2-14. 컨테이너 목록 조회

목록에서 방금 띄운 nginx를 확인할 수 있습니다.

2.3.5 자주 쓰는 명령어

다음은 Docker에서 자주 사용하는 명령어입니다.

명령어	설명
docker pull <이미지명>	Docker Hub에서 이미지 다운로드
docker images	로컬에 저장된 이미지 목록
docker logs <컨테이너ID>	컨테이너 로그 출력
docker ps -a	종료된 컨테이너 포함 전체 목록
docker stop <컨테이너ID>	실행 중인 컨테이너 종료
docker rm <컨테이너ID>	종료된 컨테이너 삭제
docker rmi <이미지ID>	이미지 삭제

지금까지 도커 명령어로 이미지를 다운로드하고 컨테이너를 직접 실행해 봤습니다. 하지만 컨테이너 내부에서 프로그램이 정상적으로 작동하고 있더라도, 외부에서 보낸 요청이 안으로 들어오지 못하면 서비스를 정상적으로 이용할 수 없습니다.

'바깥에서 보낸 요청은, 이 컨테이너 안까지 어떻게 들어오는 거지?'

컨테이너를 실제 운영 환경에서 활용하기 위해서는 외부와 데이터를 주고받는 네트워크 흐름을 이해해야 합니다.

2.4 컨테이너 통신

외부에서 보낸 요청이 컨테이너 안까지 어떻게 전달되는지, 또 컨테이너끼리 어떻게 데이터를 주고받는지 살펴보겠습니다.

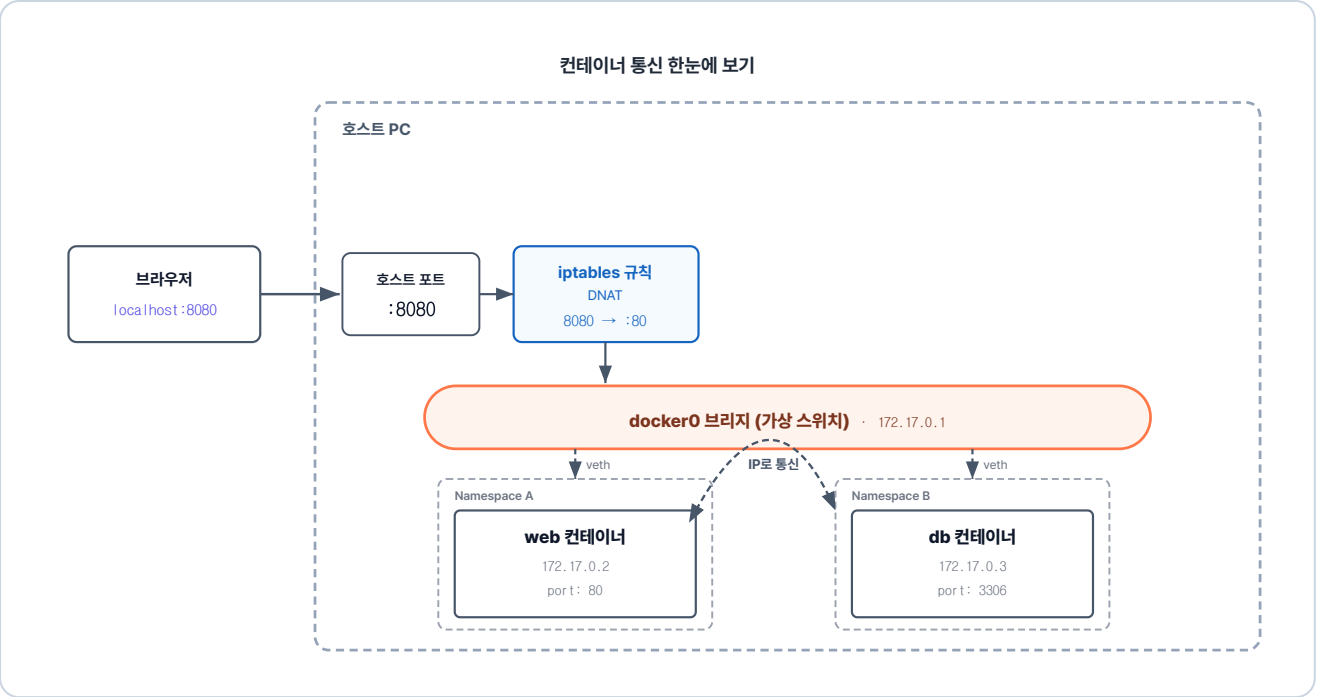


그림 2-15. 컨테이너 통신 한눈에 보기 - 외부 진입과 컨테이너끼리 IP 통신

여기서는 컨테이너 네트워크를 세 가지로 나눠 살펴보겠습니다. **컨테이너끼리 연결하는 방법, 외부 요청이 컨테이너로 들어오는 방법, 이름으로 컨테이너를 찾는 방법**입니다.

2.4.1 컨테이너 간 통신

푸드코트에 여러 식당이 있습니다. 각 식당은 서로의 주방에 들어갈 수 없습니다. 대신 각 주방의 **주방 모니터**는 **통신 케이블**로 벽에 걸린 **주문 전광판**에 연결되며, 서로의 주문 현황을 확인할 수 있습니다.

컨테이너 간 통신도 비슷합니다. 각 컨테이너는 자기만의 격리된 네트워크인 **네트워크 Namespace**를 갖고, **veth pair**라는 가상 케이블로 **docker0** 브리지에 연결됩니다. 한 컨테이너가 다른 컨테이너의 IP로 호출하면 **데이터는 docker0를 통해 전달됩니다.**

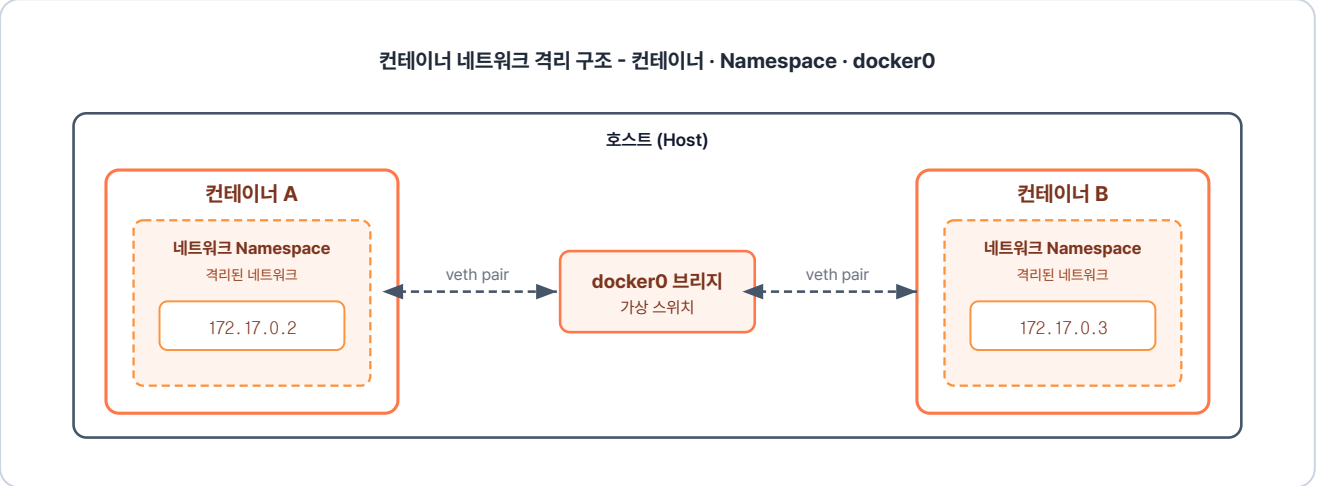


그림 2-16. 컨테이너 네트워크 격리 구조

구성 요소	하는 일
네트워크 Namespace	컨테이너마다 따로 가진, 자기 IP·포트를 쓰는 격리된 네트워크
veth pair	네트워크 Namespace를 docker0에 잇는 가상 케이블 한 쌍
docker0	같은 브리지에 묶인 컨테이너 사이로 패킷(데이터 묶음)을 중계하는 가상 브리지

2.4.2 외부에서 들어오기 - 입구 키오스크

푸드코트에서 손님은 식당으로 들어갈 수 없습니다. 대신 **키오스크**를 통해 식당에 주문을 요청합니다. 짜장면을 선택하면 A식당, 돈까스를 선택하면 B식당으로 키오스크의 규칙에 따라 주문이 전달됩니다.

외부에서 컨테이너로의 요청도 바로 접근이 불가능합니다. 그래서 키오스크의 역할을 하는 **포트포워딩**을 사용해야 합니다.

> **왜 컨테이너 IP로 바로 접근할 수 없을까**
>
> 컨테이너 IP는 실행할 때마다 새로 할당되어, 외부에서 접속할 고정된 주소가 없습니다. 게다가 Windows·macOS에서는 컨테이너가 Docker Desktop이 실행하는 리눅스 가상 머신 안에서 동작합니다. 컨테이너 IP는 이 가상 머신 내부 네트워크에만 있는 주소라, 그 바깥에 있는 호스트는 해당 IP로 도달할 수 없습니다.

예를 들어 `docker run -p 8080:80`을 실행하면 **iptables(DNAT)** 에 규칙이 작성됩니다. 그리고 외부에서 8080 포트로 요청이 오면 이 규칙에 의해 80 포트를 가진 컨테이너로 연결됩니다.

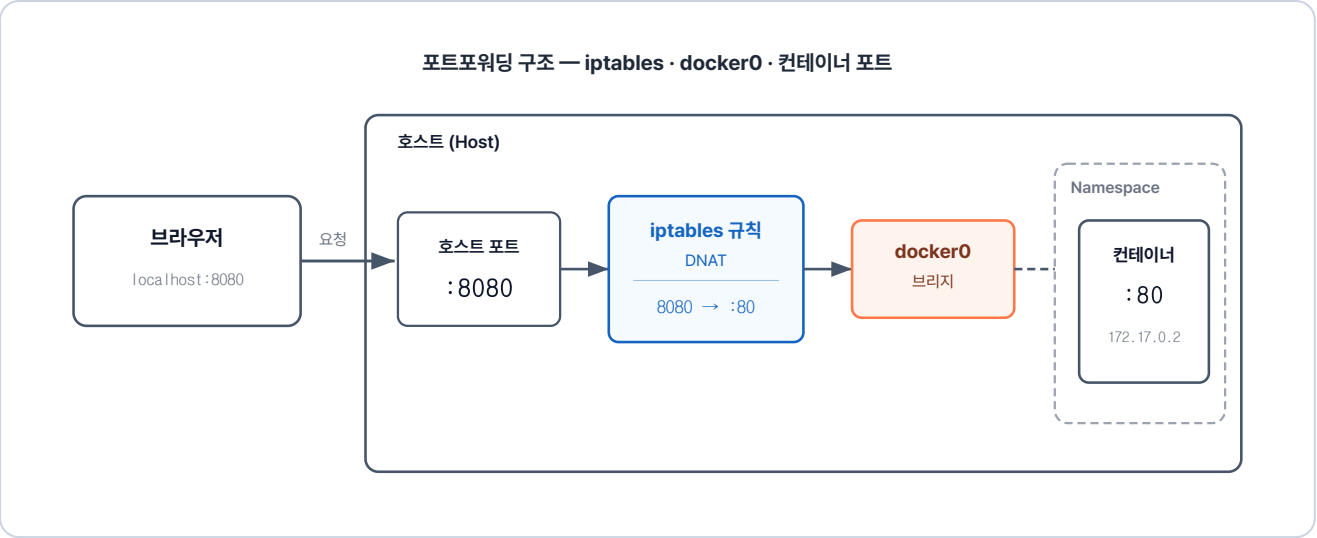


그림 2-17. 호스트 포트가 컨테이너 포트로 연결되는 구조

구성 요소	하는 일
포트포워딩	외부 공개 포트로 온 요청을 컨테이너 포트로 넘기는 메커니즘
iptables 규칙(DNAT)	외부 공개 포트로 온 요청의 목적지를 컨테이너의 IP·포트로 바꾸는 규칙
컨테이너 포트	컨테이너 안에서 앱이 듣고 있는, 요청이 최종 도착하는 포트

2.4.3 이름으로 찾기 - 홀의 안내 지도

여러 식당 중 손님은 자신이 원하는 식당의 위치를 알 수 없습니다. 이때 **안내 지도**를 보면 식당의 이름을 보고 위치를 찾을 수 있습니다.

도커에는 **Docker DNS** 기능이 있습니다. 새로운 컨테이너가 실행되면 컨테이너의 이름과 IP를 Docker DNS에 등록합니다. 그리고 **한 컨테이너가 다른 컨테이너를 이름으로 호출하면, Docker DNS가 이름에 맞는 IP를 돌려줍니다.** 이름으로 통신이 가능하기 때문에 IP가 바뀌어도 같은 이름으로 통신할 수 있습니다.

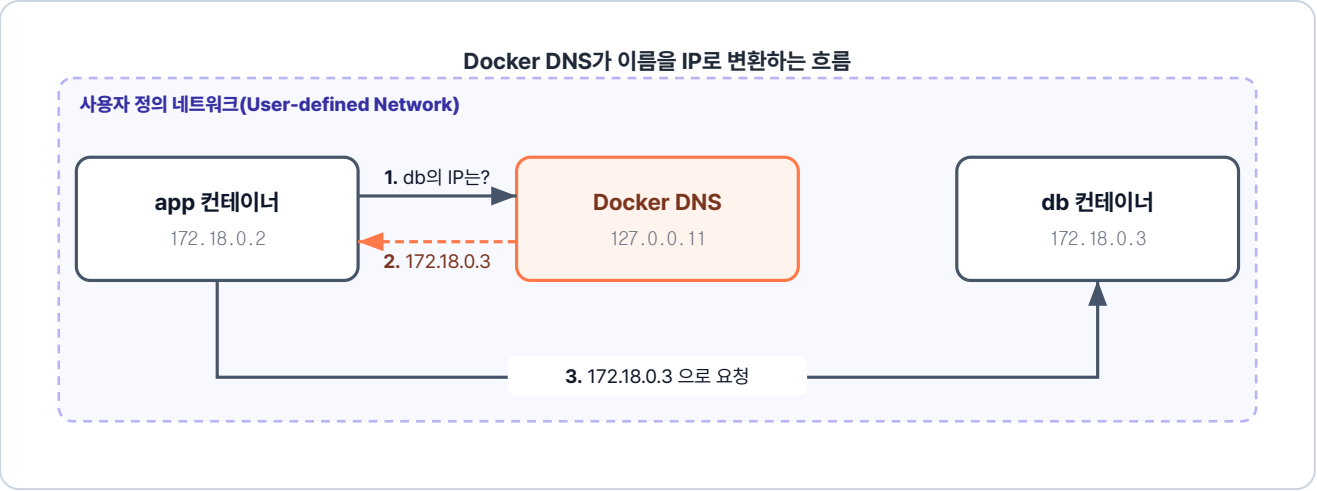


그림 2-18. Docker DNS가 이름을 IP로 변환

구성 요소	하는 일
Docker DNS	컨테이너 이름을 IP로 자동 변환해 주는 내장 DNS(Domain Name System) 서버

다만 내장 DNS는 **사용자 정의 네트워크(User-defined Network)** 에서만 동작합니다. 기본 네트워크(docker0)에서는 이름으로 찾을 수 없기 때문에, 직접 네트워크를 만들어 컨테이너들을 연결해야 합니다. 구체적인 생성 방법은 다음 챕터에서 다룹니다.

컨테이너 통신을 살펴봤으니, 이제 컨테이너 내부를 들여다볼 차례입니다. 컨테이너 안은 작은 리눅스 환경과 같아서, 가장 먼저 기본 리눅스 명령어부터 알아보겠습니다.

2.5 컨테이너 내부 - 격리된 작은 리눅스

리눅스를 연습하려면 리눅스 환경이 필요합니다. **Ubuntu**는 가장 널리 쓰이는 배포판 중 하나라 자료가 풍부하고, Docker Hub에 공식 이미지가 올라와 있어서 바로 받아 실행할 수 있습니다.

포트포워딩을 사용해 ubuntu 컨테이너 내부로 들어가 보겠습니다. 외부에서 80 포트로 요청하면 컨테이너 내부의 80 포트로 요청이 전달됩니다.

터미널 **ubuntu 컨테이너 안으로 들어가기**

```
# -i 키보드 입력 전달, -t 터미널 화면 제공, -p 80:80 포트포워딩
docker run -it -p 80:80 ubuntu
```

실행결과

```
$ docker run -it -p 80:80 ubuntu
root@c0b6dd5261d8:/#
```

그림 2-19. Ubuntu 컨테이너 안으로 들어간 상태

프롬프트가 `root@...#` 모양으로 바뀌었습니다. 방금까지 호스트의 터미널이었는데 이제는 컨테이너 안의 쉘입니다.

쉘(Shell): 사용자와 커널 사이에서 명령을 주고받는 프로그램입니다. 사용자가 쉘에 명령을 내리면 쉘이 커널에 그 명령을 전달하고 커널이 프로세스를 만들어 실제 작업을 수행합니다.

2.5.1 자주 쓰는 리눅스 명령어

자주 쓰는 리눅스 명령어는 다음과 같습니다.

용도	명령	설명
위치 확인	**pwd**	현재 위치
이동	**cd <경로>**	폴더 이동
목록	**ls, ls -la**	파일/폴더 목록 (숨김 포함)
폴더 생성	**mkdir <이름>**	폴더 만들기
파일 생성	**touch <이름>**	빈 파일
복사	**cp <원본> <사본>**	파일 복사
이동/이름변경	**mv <원본> <대상>**	이동 또는 rename
삭제	**rm <파일>, rm -r <폴더>**	삭제
패키지 설치	**apt update && apt install -y <이름>**	패키지 설치
프로세스	**ps -ef, kill <PID>**	실행 중 프로세스 / 종료
검색	**find <경로> -name <이름>**	파일 검색

용도	명령	설명
로그	**tail -n <수> <파일>**	마지막 N줄

경로를 쓸 때는 **절대 경로**와 **상대 경로**를 구분해야 합니다.

> **절대 경로 / 상대 경로**
> 둘의 차이는 **어디서부터 시작해서 읽느냐**에 있습니다.
>
> - **절대 경로**는 **루트 경로(/)** 부터 시작해서 목적지까지의 전체 주소를 명시합니다. 현재 위치가 어디든 같은 곳을 가리킵니다. 리눅스에서는 결과적으로 `/` 로 시작하는 형태가 됩니다. 예: `/bin` , `/home/ubuntu/docs`
> - **상대 경로**는 **현재 디렉터리**를 기준으로 목적지까지의 경로를 나타냅니다. 현재 위치에 따라 같은 표기가 다른 곳을 가리킬 수 있습니다. 예: `bin` (현재 디렉터리 아래 bin), `../etc` (상위 디렉터리의 etc)
> - 경로에서 `.` 은 **현재 디렉터리**, `..` 은 **상위 디렉터리**를 뜻합니다.

2.5.2 실습: 컨테이너 안에 nginx 깔아보기

앞서 이미지로 실행했던 nginx를, 이번엔 컨테이너 안에서 직접 설치해 보겠습니다. 컨테이너는 최소 구성이라 필요한 패키지는 직접 설치해야 합니다.

터미널 **컨테이너 내부 - 패키지 목록 갱신**

```
apt update           # 최신 패키지 목록으로 업데이트
apt install -y nginx # nginx 설치
nginx                # nginx 실행
```

실행 결과

```
root@c0b6dd5261d8:/# apt update
Hit:1 http://security.ubuntu.com/ubuntu noble-security InRelease
Hit:2 http://archive.ubuntu.com/ubuntu noble InRelease
Hit:3 http://archive.ubuntu.com/ubuntu noble-updates InRelease
Hit:4 http://archive.ubuntu.com/ubuntu noble-backports InRelease
7 packages can be upgraded. Run 'apt list --upgradable' to see them.
root@c0b6dd5261d8:/# apt install -y nginx
Need to get 521 kB of archives.
After this operation, 1352 kB of additional disk space will be used.
Fetched 521 kB in 2s (217 kB/s)
Selecting previously unselected package nginx.
Preparing to unpack .../nginx_1.24.0-2ubuntu7.6_amd64.deb ...
Unpacking nginx (1.24.0-2ubuntu7.6) ...
Setting up nginx (1.24.0-2ubuntu7.6) ...
root@c0b6dd5261d8:/# nginx
```

그림 2-20. nginx 설치와 실행 결과

설치 로그가 마무리되며 Nginx 설치가 끝났습니다.

네트워크 확인 도구인 `net-tools`를 추가로 설치하고 Nginx가 어느 포트에서 대기 중인지 확인합니다.

터미널 **컨테이너 내부 - net-tools 설치**

```
apt install -y net-tools   # net-tools: 네트워크 확인 도구 모음
netstat -nlpt              # 열려 있는 TCP 포트 확인
```

실행 결과

```
root@c0b6dd5261d8:/# netstat -nlpt
Active Internet connections (only servers)
Proto Recv-Q Send-Q Local Address Foreign Address State PID/Program name
tcp 0 0 0.0.0.0:80 0.0.0.0:* LISTEN 348/nginx: master p
tcp6 0 0 :::80 :::* LISTEN 348/nginx: master p
```

그림 2-21. 포트 상태 확인

80 포트가 열려 있는 것을 확인합니다. 다음은 브라우저로 접속할 차례입니다. 주소창에 `localhost:80`을 칩니다.



그림 2-22. nginx 환영 페이지 응답

nginx 환영 페이지가 떴습니다. 컨테이너를 실행할 때 포트포워딩으로 포트를 열어 둔 덕에, 컨테이너 안의 nginx와 브라우저가 연결됐기 때문입니다.

2.5.3 vim으로 파일 편집

서버 설치를 마쳤더라도 운영하다 보면 설정 파일을 수정해야 할 일이 생기기 마련입니다. 리눅스에서 파일을 편집하려면 텍스트 편집기가 필요한데, 여기서는 널리 쓰이는 **Vim**을 설치합니다.

터미널 **컨테이너 내부 - vim 설치**

```
apt install -y vim   # 터미널 편집기 vim 설치
```

> **설치 중 시간대 선택 화면**
> apt install vim 도중 Geographic area와 Time zone을 선택하는 화면이 나옵니다. 이때 **Asia**, **Seoul**을 각각 선택합니다.

vim의 사용 명령어는 다음과 같습니다.

단계	동작	키
1	파일 열기/생성	**vim <파일명>**

단계	동작	키
2	입력 모드	**i**
3	일반 모드로	**ESC**
4	저장 후 종료	**:wq**

vim으로 간단한 파일을 만들어 봅니다.

터미널 **컨테이너 내부 - vim으로 파일 생성**

```
vim test1.txt   # test1.txt 파일 생성
```

`i`를 눌러 입력 모드로 전환하고 내용을 작성한 뒤 `ESC` → `:wq`로 저장합니다.

실행결과

```
hello docker
~
~
~
~
:wq
```

그림 2-23. vim 편집 화면

저장이 잘 됐는지 `cat`으로 내용을 확인합니다.

터미널 **컨테이너 내부 - 파일 내용 확인**

```
cat test1.txt   # 파일 내용 출력
```

실행결과

```
root@c0b6dd5261d8:/# cat test1.txt
hello docker
```

그림 2-24. 파일 내용 출력

실습을 마친 후 컨테이너 안에서 빠져나오기 위해 `exit`를 입력합니다. 그렇게 터미널로 돌아온 뒤 `docker ps`로 실행 중인 컨테이너를 확인하면, 방금까지 들어가 있던 ubuntu가 목록에 없습니다.

'컨테이너를 빠져나왔는데 같이 종료가 되네.'

방금처럼 컨테이너가 언제 종료되고 언제 계속 실행되는지 이해하려면, 그 중심에서 도는 **메인 프로세스**를 알아야 합니다.

2.6 컨테이너의 메인 프로세스

2.6.1 메인 프로세스란?

메인 프로세스는 도커 컨테이너가 돌리는 중심 프로그램으로, 컨테이너 안에서 가장 먼저 실행되어 **프로세스 ID(PID) 1번**을 받습니다. 자동차가 엔진이 도는 동안에만 달릴 수 있듯, 컨테이너 역시 **이 메인 프로세스가 멈추면 그 즉시 함께 종료됩니다**.

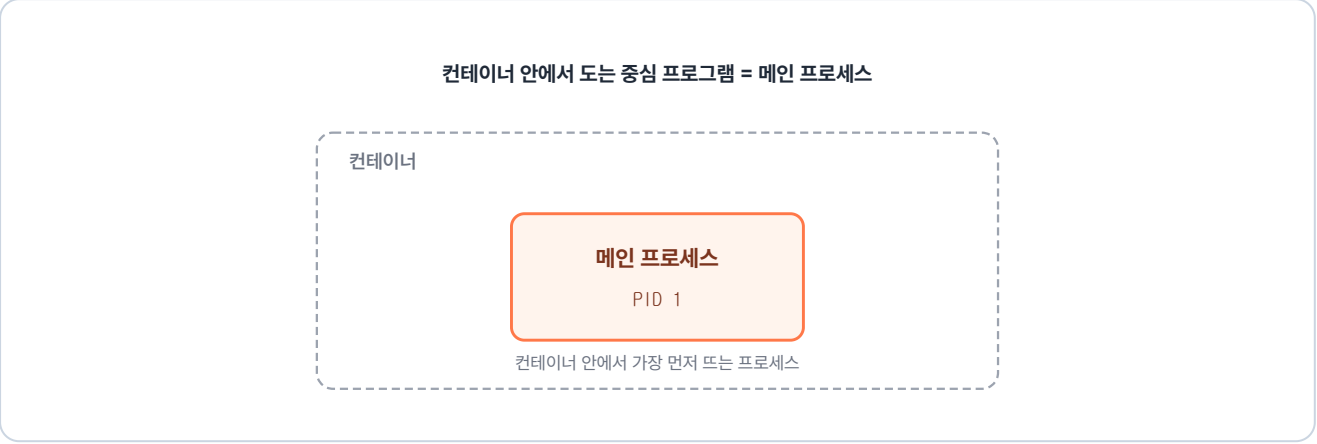


그림 2-25. 컨테이너 안에서 도는 중심 프로그램이 메인 프로세스입니다

2.6.2 메인 프로세스에서 빠져나오기

`exit`를 입력했을 때 컨테이너가 종료된 상황을 한 번 더 재연해 보겠습니다. 컨테이너에 `dead`라는 이름을 붙여 실행한 뒤, 안에서 `exit`로 빠져나옵니다.

터미널 **exit로 빠져나오는 시나리오**

```
docker run -it --name dead ubuntu   # 실행 후 exit로 나옴
exit
```

```
docker ps                                   # 종료 여부 확인
```

실행결과

```
root@f9a2b3c1d4e5:/# exit
exit
$ docker ps
CONTAINER ID IMAGE COMMAND CREATED STATUS PORTS NAMES
```

그림 2-26. exit로 빠져나오면 컨테이너가 종료

목록에 dead는 없습니다. `exit`**가 메인 프로세스를 종료하면서 컨테이너까지 함께 멈췄기 때문입니다.** 그런데 컨테이너를 빠져나오는 방법이 `exit`만 있는 것은 아닙니다.

`CTRL+P → CTRL+Q`를 누르면 메인 프로세스는 그대로 둔 채 터미널 연결만 끊을 수 있습니다. `alive`라는 이름으로 컨테이너를 실행한 뒤, 이 키로 빠져나옵니다.

터미널 **CTRL+P → CTRL+Q로 빠져나오는 시나리오**

```
docker run -it --name alive ubuntu   # CTRL+P → CTRL+Q로 나옴
docker ps                            # 종료 여부 확인
```

실행결과

```
root@17943f60b5cd:/#
$ docker ps
CONTAINER ID IMAGE COMMAND CREATED STATUS PORTS NAMES
17943f60b5cd ubuntu "/bin/bash" Less than a second ago Up Less than a second alive
```

그림 2-27. CTRL+P → CTRL+Q로는 컨테이너가 유지

이번에는 alive가 목록에 남아 있습니다. `CTRL+P → CTRL+Q`**는 메인 프로세스를 건드리지 않고 터미널 연결만 끊기 때문에 컨테이너는 그대로 살아 있습니다.**

2.6.3 메인 프로세스가 유지되는 조건

컨테이너가 내부 프로세스를 실행하는 방식은 이미지마다 다릅니다. 앞서 nginx를 `-d`로 백그라운드에서 실행했습니다. 같은 방식을 ubuntu에도 적용해 두 이미지가 어떻게 다른지 비교해 보겠습니다.

터미널 **ubuntu를 -d만으로 실행**

```
docker run -d ubuntu   # ubuntu를 백그라운드로 실행
docker ps              # 실행 중인 컨테이너 목록 확인
```

실행결과

```
$ docker run -d ubuntu
0b74e66f5b0f3f4cb30b353503f2ef0c162e63f19be512e938f28e07d01dc1e7
$ docker ps
CONTAINER ID IMAGE COMMAND CREATED STATUS PORTS NAMES
```

그림 2-28. ubuntu는 -d만으로는 바로 종료

'nginx는 백그라운드에서 실행이 됐는데, ubuntu는 안 되는 건가?'

ubuntu는 백그라운드로 실행되자마자 종료되어 실행 목록에 보이지 않습니다. 같은 -d 옵션인데 nginx와 결과가 다른 이유는 이미지마다 메인 프로세스가 다르기 때문입니다.



그림 2-29. 같은 PID 1 자리에 이미지마다 다른 프로그램이 메인 프로세스로 들어갑니다

ubuntu가 바로 종료된 이유: ubuntu 이미지의 메인 프로세스는 사용자의 명령을 입력받는 **셸(bash)** 입니다. `-it` 옵션을 주어 터미널을 연결하면 입력을 기다리기 위해 컨테이너가 계속 유지되지만, `-d` 옵션만 주면 입력을 받을 터미널이 없어져 버립니다. 즉, 셸이 더 이상 할 일이 없다고 판단하여 컨테이너가 그 즉시 종료됩니다.

nginx와의 차이: 반면 웹 서버인 nginx의 목적은 사용자의 입력을 받는 것이 아니라 외부에서 들어오는 **웹 요청**을 기다리는 것입니다. 터미널이 없어도 **요청 대기**라는 명확한 할 일이 멈추지 않으므로 종료되지 않고 계속 떠 있습니다.

ubuntu처럼 `-d` 만으로는 바로 종료되는 이미지는, `-dit` 옵션으로 터미널을 함께 열어야 백그라운드에서 계속 실행됩니다.

결국 컨테이너가 백그라운드에서 계속 살아 있으려면, **메인 프로세스가 멈추지 않고 실행되고 있어야 합니다.** 그러려면 두 가지 방법이 있습니다. **메인 프로세스가 입력을 기다리도록 터미널을 함께 열어 두거나, 메인 프로세스를 계속 실행되는 다른 프로그램으로 바꾸면 됩니다.**

2.6.4 메인 프로세스 바꾸기 - CMD

이번에는 `docker run <이미지> <CMD 옵션>` 형태로 메인 프로세스를 직접 바꿔 보겠습니다. ubuntu의 기본 메인 프로세스인 bash 대신, 1000초 동안 멈춰 있는 **sleep 1000** 을 지정합니다.

터미널 **sleep을 메인 프로세스로 실행**

```
docker run -d ubuntu sleep 1000   # bash 대신 sleep을 메인으로
docker ps                         # 1000초 동안 살아 있음
```

실행결과

```
$ docker run -d ubuntu sleep 1000
7aca4e791643d3a39917aecc97868a62f5c1f6fa3df1ac63248e703650835cd3
$ docker ps
CONTAINER ID IMAGE COMMAND CREATED STATUS PORTS NAMES
7aca4e791643 ubuntu "sleep 1000" Less than a second ago Up Less than a second goofy_bose
```

그림 2-30. CMD를 sleep으로 바꾸면 유지

이제 ubuntu가 목록에 남아 있습니다. 메인 프로세스가 bash에서, 터미널이 없어도 1000초 동안 멈춰 있는 sleep으로 바뀌었기 때문입니다. 이렇게 **기본 메인 프로세스를 바꾸는 설정을 CMD**라고 합니다.

2.6.5 실행 중인 컨테이너에 들어가기 - attach와 exec

이제 백그라운드로 실행 중인 컨테이너 안으로 들어가 보겠습니다. 들어가는 명령은 **attach**와 **exec**, 두 가지입니다.

먼저 attach를 알아보겠습니다. attach는 **메인 프로세스에 직접 연결**하는 방식입니다.

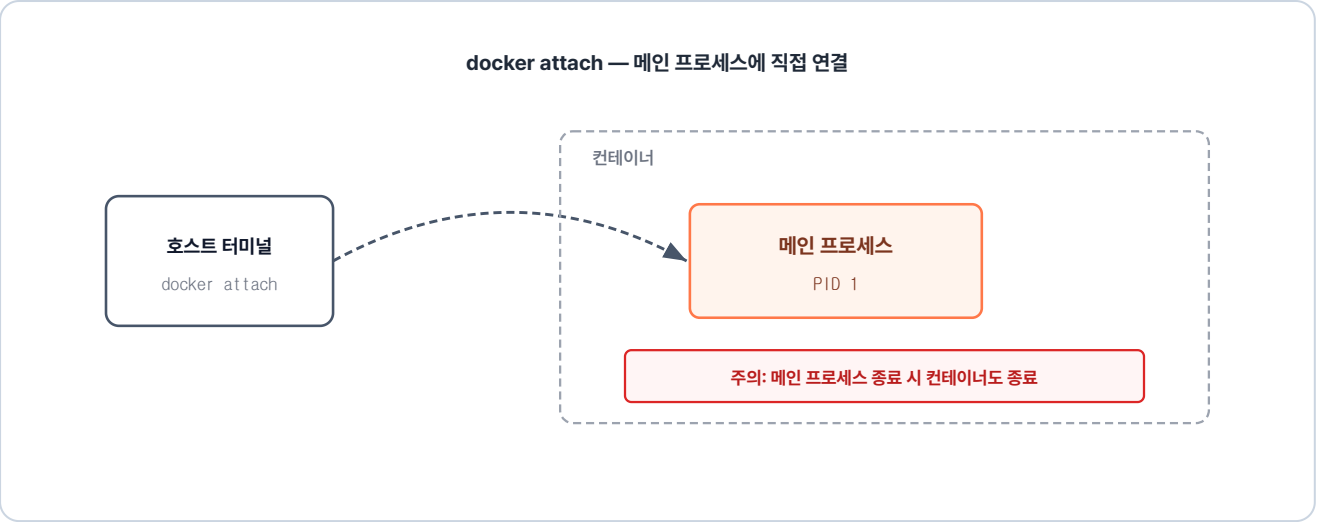


그림 2-31. attach는 이미 떠 있는 PID 1 프로세스에 터미널을 직접 연결하는 방식

확인하기 위해 ubuntu를 `-dit`로 백그라운드에서 실행합니다. 실행 중인 컨테이너의 ID를 확인한 뒤 `docker attach <컨테이너ID>`로 접속합니다.

터미널 **-dit 조합으로 ubuntu 살려 두기**

```
docker run -dit ubuntu  # -d(백그라운드)와 -it(터미널)를 합쳐 bash 유지
docker ps               # 실행중인 컨테이너 확인
docker attach d2b1      # attach로 접근
```

실행결과

```
$ docker run -dit ubuntu
d2b18acc2cf05a40644f0ca8536cb50a0ff560a1be0c9ce2fbf5ddbbba0e729c
$ docker ps
CONTAINER ID IMAGE COMMAND CREATED STATUS PORTS NAMES
d2b18acc2cf0 ubuntu "/bin/bash" Less than a second ago Up Less than a second angry_engelbart
$ docker attach d2b1
root@d2b18acc2cf0:/#
```

그림 2-32. attach로 접근

메인 프로세스인 bash의 터미널이 그대로 나타났습니다. 메인 프로세스에 연결됐으므로, 여기서 `exit`로 나오면 그 bash가 끝나고 컨테이너도 멈춥니다. 그래서 컨테이너를 살려 둔 채 빠져나오려면 `CTRL+P → CTRL+Q`를 써야 합니다.

이번에는 exec를 알아보겠습니다. exec는 **컨테이너 안에 새 프로세스를 실행**하는 방식입니다.

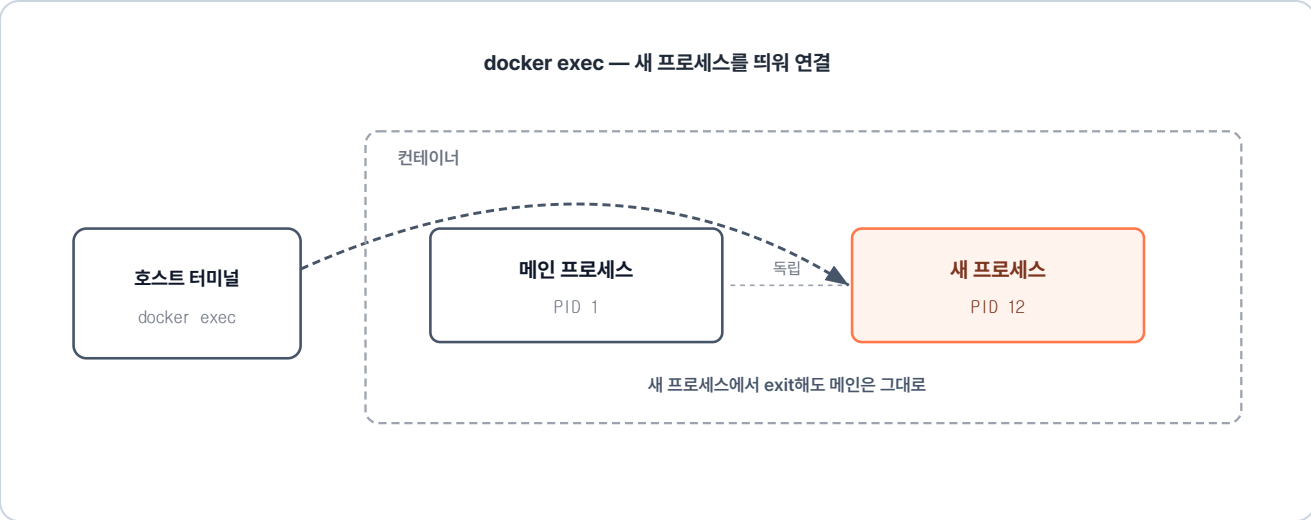


그림 2-33. exec는 컨테이너 안에 새 bash 프로세스를 띄우고 내 터미널에 연결하는 방식

새 프로세스에 연결되므로 exit해도 메인 프로세스는 그대로입니다. **exec는 새 프로세스가 실행되기 때문에 프로세스를 지정해야 합니다.** `docker exec -it <컨테이너ID> <프로세스>` 형태로 접근합니다.

터미널 **실행 중 컨테이너에 새 bash 접속**

```
docker exec -it d2b1 bash   # 실행 중인 컨테이너에 새 bash 접속
```

실행결과

```
$ docker exec -it d2b1 bash
root@d2b18acc2cf0:/#
```

그림 2-34. exec로 접근

attach와 똑같이 bash 터미널이 나타나서, 화면만 보면 둘이 같아 보입니다. exec가 정말 새 프로세스를 실행했는지 눈으로 확인해 보겠습니다. **새로운 터미널 창을 연 후,** `ps aux` 명령으로 컨테이너 안에서 실행 중인 프로세스 목록을 조회합니다.

터미널 **컨테이너 내부 프로세스 목록 확인**

```
docker exec d2b1 ps aux   # 컨테이너 내부 프로세스 목록 확인
```

프로세스 목록 확인

```
$ docker exec d2b1 ps aux
USER PID %CPU %MEM VSZ RSS TTY STAT START TIME COMMAND
root 1 0.0 0.0 4588 3840 pts/0 S+ 03:29 0:00 /bin/bash
root 12 0.1 0.0 4588 3840 pts/1 S+ 03:34 0:00 bash
root 21 63.6 0.0 7888 4096 ? Rs 03:34 0:00 ps aux
```

그림 2-35. 프로세스 목록 확인

목록에 bash가 둘 있습니다. **PID 1**은 앞서 본 메인 프로세스이고, **PID 12**가 방금 exec로 새로 실행한 프로세스입니다. 두 프로세스는 같은 컨테이너의 파일과 네트워크를 공유하지만, 각자 독립적으로 동작합니다.

'결국 컨테이너를 제어한다는 건, 그 안에서 실행되는 메인 프로세스를 다루는 일이었구나.'

2.7 commit - 실행 중인 컨테이너를 이미지로

앞에서 컨테이너 안에 패키지를 설치하고 파일도 직접 만들어 봤습니다. 그런데 컨테이너를 내리면 이렇게 작업한 내용이 모두 사라집니다.

'내가 수정한 이 상태를 기록해두려면 어떻게 해야 하지?'

컨테이너의 현재 상태를 그대로 이미지로 저장하는 명령이 `docker commit`입니다. 이번엔 직접 만든 이미지를 Docker Hub에 올려보겠습니다.

2.7.1 Tomcat으로 commit 실습하기

나만의 이미지를 만들어보겠습니다. 우선 Tomcat 이미지를 받아 컨테이너를 실행합니다.

터미널 **Tomcat 컨테이너 실행**

```
docker run -d -p 8080:8080 tomcat   # 8080 포트로 Tomcat 백그라운드 실행
```

실행 결과

```
$ docker run -d -p 8080:8080 tomcat
5fcd8e3a1c9b07d4f2e6a5083b1d9c4f7e206a8d3b5f1e9c0a7d4b2f86e1c3a0
```

그림 2-36. Tomcat 백그라운드 실행 결과

실행 후 브라우저에서 **localhost:8080** 으로 접속하면 **404**가 뜹니다.



그림 2-37. Tomcat 404 에러 화면

'연결은 된 거 같은데 왜 404 에러가 뜨지?'

Tomcat은 루트 주소로 접속하면 `webapps/ROOT/` 폴더의 index.html을 기본 페이지로 응답합니다. 그런데 Docker Hub에서 받은 이미지는 `webapps/`가 비어 있어, 이 index.html이 없으니 404가 뜹니다.

그래서 `webapps/ROOT/`에 index.html을 직접 만들어 브라우저에서 응답되도록 해 보겠습니다.

터미널 **Tomcat 컨테이너 접속과 설정 수정**

```
docker exec -it 5fcd bash              # 컨테이너 안에 새 bash 터미널(입력창)을 열어 접속
cd /usr/local/tomcat/webapps           # Tomcat의 웹앱 기본 경로로 이동
mkdir ROOT && cd ROOT                  # ROOT 폴더 생성 후 그 안으로 이동
apt update && apt install -y vim       # 패키지 목록 갱신 후 vim 편집기 설치
vim index.html                         # 파일 만들고 HTML 작성 후 저장
```

vim으로 index.html을 만들어 저장한 뒤 **localhost:8080** 으로 다시 접속하면 방금 만든 페이지가 응답됩니다.



그림 2-38. index.html 응답 확인

확인이 끝나면 `exit`를 입력해 컨테이너를 빠져나옵니다. `exec`로 컨테이너에 접속했기 때문에 `exit`를 입력해도 메인 프로세스는 유지됩니다.

이제 `docker commit <컨테이너ID> <본인-dockerhub-id>/<이미지이름>` 형식으로 명령어를 입력해 현재 상태를 기록합니다.

터미널 컨테이너 상태를 새 이미지로 저장

```
docker commit 5fcd coderyu5523/tomcat   # 현재 컨테이너 상태를 새 이미지로 저장
```

실행결과

```
$ docker commit 5fcd coderyu5523/tomcat
sha256:0f8181585ca9e7f2b68c623f0ce6d64f5fddfeb19076c7c0e82e55ca4629cfbb
```

그림 2-39. 이미지 커밋 완료 화면

저장된 이미지가 목록에 잘 들어갔는지 확인합니다.

터미널 로컬 이미지 목록 확인

```
docker images   # 로컬에 저장된 이미지 목록 확인
```

실행 결과

```
$ docker images
REPOSITORY TAG IMAGE ID CREATED SIZE
coderyu5523/tomcat latest 0f8181585ca9 12 seconds ago 541MB
tomcat latest f1e6c9d4a7b8 2 weeks ago 489MB
```

그림 2-40. 로컬 이미지 목록에 새 이미지 추가 확인

2.7.2 Docker Hub에 올리기

지금까지 저장한 이미지는 로컬에만 있습니다. 이 이미지를 Docker Hub에 올려두면 다른 PC나 서버에서 내려받을 수 있습니다.

Docker Hub에 로그인한 뒤 `docker push <본인-dockerhub-id>/<이미지이름>` 형식으로 이미지를 올립니다.

터미널 **Docker Hub 로그인과 푸시**

```
docker login                      # Docker Hub 로그인
docker push coderyu5523/tomcat    # 로컬 이미지를 Hub로 업로드
```

실행 결과

```
$ docker push coderyu5523/tomcat
The push refers to repository [docker.io/coderyu5523/tomcat]
247bb26cb831: Preparing
817807f3c64e: Preparing
6e713be20fa2: Preparing
cc1e0a391268: Preparing
de9be28b9519: Preparing
c318c44e952a: Preparing
4f4fb700ef54: Preparing
5dd5756e7123: Preparing
cc1e0a391268: Waiting
```

그림 2-41. docker push 실행 결과

업로드가 완료된 뒤 Docker Hub의 Repositories 탭을 열면 방금 올린 이미지를 확인할 수 있습니다.

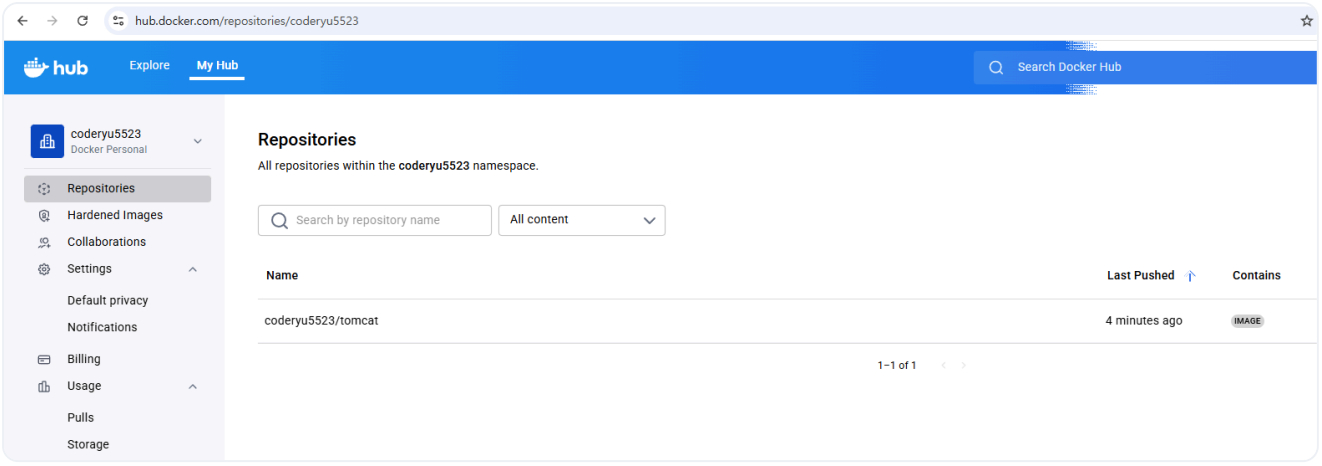


그림 2-42. Docker Hub 저장소 확인

2.8 마운트로 데이터 보존

방금 한 commit은 그 시점의 컨테이너를 이미지로 만들 뿐, 데이터를 계속 보관하는 저장소는 아닙니다. 컨테이너가 실행되며 저장되는 데이터는 이미지에 남지 않아, 컨테이너를 삭제하면 데이터도 통째로 날아갑니다.

'컨테이너를 내려도 데이터는 남아 있어야 하는데.'

데이터를 잃지 않으려면 컨테이너 외부에 보관해야 합니다. 컨테이너와 외부 저장소를 연결해 주는 기능이 **마운트**입니다. 마운트 방식은 두 가지입니다.

2.8.1 바인드 마운트: 호스트 폴더와 직접 연결

첫 번째 방식은 **호스트 PC의 특정 폴더**를 **컨테이너 내부의 경로**와 연결하는 **바인드 마운트(Bind Mount)**입니다. USB를 컴퓨터에 꽂으면 그 안의 파일을 그대로 볼 수 있듯, 컨테이너도 호스트 PC의 폴더를 꽂아서 쓰는 방식입니다.

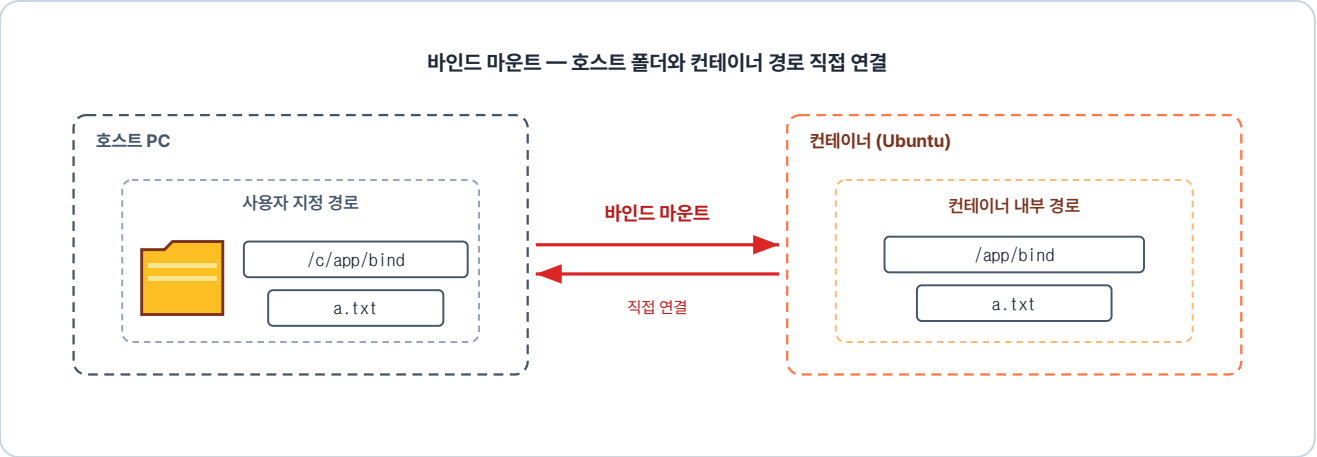


그림 2-43. 호스트 폴더와 컨테이너 경로가 같은 데이터를 바라보도록 묶인 상태

이제 터미널을 열고 직접 해보겠습니다. 호스트의 `/c/app/bind` 경로에 폴더를 하나 만들고, 컨테이너 안의 경로와 연결합니다.

터미널 **bind mount 폴더 준비와 마운트 실행**

```
# 호스트 PC에 공유할 폴더 생성
mkdir -p /c/app/bind
# 문법: docker run -it --mount type=bind,src=<호스트 경로>,dst=<컨테이너 경로> <이미지>
# 호스트 폴더(src)와 컨테이너 경로(dst)를 연결
docker run -it --mount type=bind,src=/c/app/bind,dst=/app/bind ubuntu
```

실행결과

```
$ docker run -it --mount type=bind,src=/c/app/bind,dst=/app/bind ubuntu
root@a3f1e2b4c5d6:/#
```

그림 2-44. 바인드 마운트로 ubuntu 실행

컨테이너 안 `/app/bind`에 `a.txt`를 만들고, 호스트 쪽 `/c/app/bind`에서 같은 파일이 보이는지 확인합니다.

터미널 **bind mount 양방향 동기화 확인**

```
touch /app/bind/a.txt   # 컨테이너 안에서 파일 생성
```

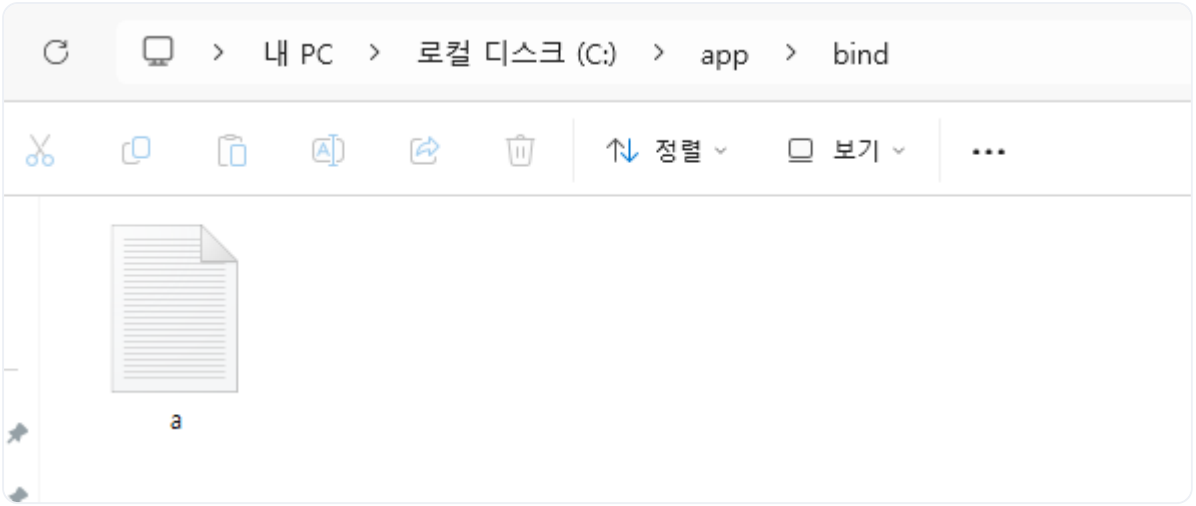


그림 2-45. 호스트 PC에서 같은 파일이 보이는 것을 확인

2.8.2 볼륨 마운트: Docker가 관리하는 저장소

바인드 마운트는 **사용자가 폴더를 직접 관리**해야 합니다. 반대로 **볼륨 마운트(Volume Mount)** 는 **Docker가 자동으로 관리**합니다. **사용자는 볼륨에 이름만 정하면, 호스트의 실제 저장 위치는 Docker가 알아서** 처리합니다.

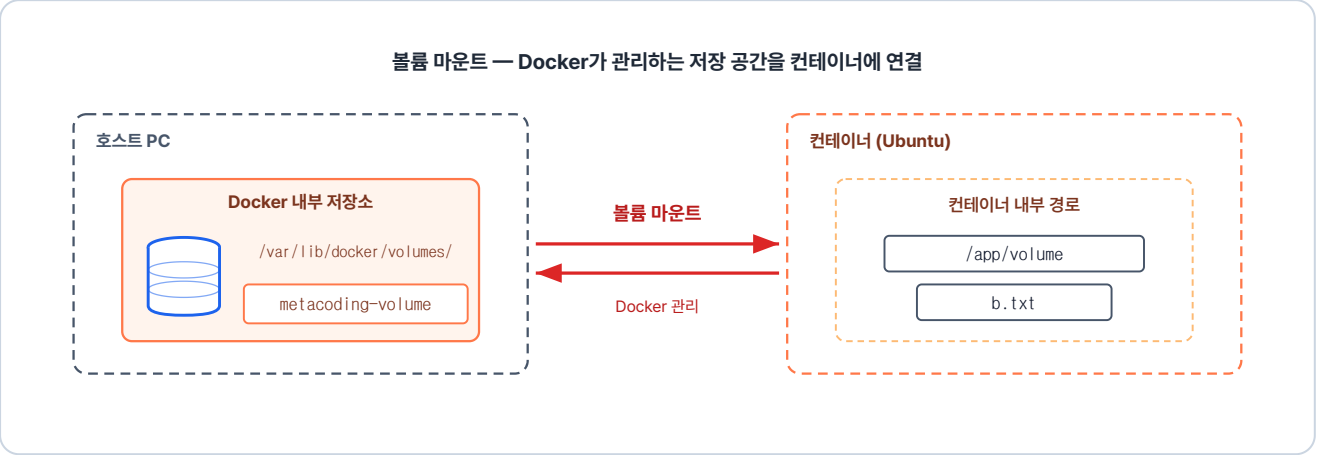


그림 2-46. Docker 엔진이 관리하는 내부 저장 공간에 데이터가 저장되는 구조

이제 직접 해보겠습니다. 다음 명령은 `metacoding-volume` 이라는 볼륨을 만들어 ubuntu 컨테이너를 실행합니다.

터미널 **named volume 마운트로 ubuntu 실행**

```
# 문법: docker run -it --mount type=volume,src=<볼륨 이름>,dst=<컨테이너 경로> <이미지>
# 볼륨 마운트로 ubuntu 실행
docker run -it --mount type=volume,src=metacoding-volume,dst=/app/volume ubuntu
```

실행결과

```
$ docker run -it --mount type=volume,src=metacoding-volume,dst=/app/volume ubuntu
root@d6f096d57b3e:/#
```

그림 2-47. 볼륨 마운트로 ubuntu 실행

데이터는 Docker가 관리하는 저장소에 보관되며, 컨테이너는 `/app/volume` 경로를 통해 이 데이터를 읽고 씁니다. 따라서 해당 경로에 저장한 데이터만 유지되고, 그 외의 경로에 저장된 파일은 컨테이너 삭제 시 함께 지워집니다.

이 폴더에 `b.txt`를 만들고, 컨테이너를 빠져나와 볼륨이 그대로 남아 있는지 확인합니다.

터미널 **볼륨에 파일 생성**

```
touch /app/volume/b.txt    # 볼륨 안에 빈 파일 생성
exit                       # 컨테이너 종료
docker volume ls           # 볼륨 목록 재확인
```

실행결과

```
root@d6f096d57b3e:/# exit
exit
$ docker volume ls
DRIVER VOLUME NAME
local metacoding-volume
```

그림 2-48. 컨테이너가 사라져도 볼륨은 유지

컨테이너는 사라졌지만 볼륨은 목록에 그대로 남아 있습니다. 이어서 같은 볼륨을 연결해 새 컨테이너를 다시 실행합니다.

터미널 **같은 볼륨으로 새 컨테이너에서 데이터 확인**

```
# 같은 볼륨으로 새 컨테이너 실행
docker run -it --mount type=volume,src=metacoding-volume,dst=/app/volume ubuntu
ls /app/volume
```

실행 결과

```
$ docker run -it --mount type=volume,src=metacoding-volume,dst=/app/volume ubuntu
root@e8a3f1c47b29:/#
root@e8a3f1c47b29:/# ls /app/volume
b.txt
```

그림 2-49. 볼륨 데이터 재사용

방금 만든 `b.txt`가 새 컨테이너 안에서도 그대로 보입니다. **컨테이너를 내려도 데이터는 사라지지 않습니다**.

다음 챕터로 넘어가기 전, 실행 중인 컨테이너를 모두 종료합니다.

터미널 **실습한 컨테이너 전체 종료**

```
docker stop $(docker ps -q)   # 실행 중인 모든 컨테이너 종료
```

오픈이는 하루 사이에 익힌 키워드를 정리했습니다. 격리, 이미지, 포트포워딩, CMD, 볼륨이 머릿속에 정리됐습니다. 어제 헤맸던 그 에러도 이제는 어디에서 막혔던 건지 짚을 수 있을 것 같았습니다.

다만 지금까지 익힌 건 도커와 컨테이너의 기본입니다. 실제 현장에서 그대로 쓰기에는 한계가 있습니다.

'기본은 익혔다. 그런데 실제 현장은 이것만으로는 부족하겠지.'

다음 챕터에서는 컨테이너를 활용해 실제 서비스를 구성하는 방법을 익혀 보겠습니다.

이것만은 기억하자

- → **컨테이너는 격리된 프로세스입니다.** 파일시스템·네트워크·프로세스 공간만 따로 나뉘고, 커널은 호스트와 공유합니다.
- → **이미지는 설계도, 컨테이너는 찍혀 나온 결과입니다.** 붕어빵 틀 하나로 여러 붕어빵을 찍듯, 이미지 하나로 여러 컨테이너를 찍어냅니다.
- → **컨테이너 통신은 docker0가 중심입니다.** 컨테이너끼리는 docker0로 연결되고, 외부 요청은 포트포워딩으로 들어옵니다.
- → **메인 프로세스가 살아 있어야 컨테이너도 살아 있습니다.** 그래서 이미지의 CMD에 실행할 프로그램을 미리 지정해 둡니다.
- → **마운트는 컨테이너와 외부 저장소를 잇습니다.** 바인드는 호스트의 특정 폴더를, 볼륨은 Docker 엔진이 관리하는 저장소를 컨테이너에 연결합니다.